

Template

Initial Proposal Template

Project Title: TRYBE: Trust Your Vybe -- A BPI Banking Partner

Track Selection

(Please select one)

- Digitalization
- ESG+E2: Financial Inclusion & Sustainable Business Growth
- **Hyper-Personalization & Customer Experience**
- Workplace productivity & Future of Work
- Synergies & Ecosystem Collaboration

Participant Information

- **Team Name:** vybeCoders
- **Team Members:**
 - Name 1: Darius Ardales
 - Name 2: Ronnel Bermas
 - Name 3: Camille Silvio
 - Name 4: Christine Sotoza
- **Contact email of Team Members:**
 - Email 1: darius.vincent5@gmail.com
 - Email 2: ronnel.bermas@student.ateneo.edu
 - Email 3: camille.silvio@student.ateneo.edu
 - Email 4: christine.sotoza@student.ateneo.edu

Project Overview

1. Problem Statement

Problem Statement:

BPI Vybe users frequently encounter “floating cash” issues, where debited funds fail to promptly transfer money from one merchant to another, leading to customer dissatisfaction and diminished trust. The absence of proactive, real-time transaction monitoring undermines the reliability and reputation of the digital banking platform resulting in a Reddit thread entitled “BPI Vybe - Most unreliable online banking app.”

Context:

One of the key impacts the pandemic has had on this generation is the increased emphasis

on the online world. This digital shift extends beyond online shopping or selling and now encompasses essential services such as banking. Online banking has gained prominence, particularly due to its no-contact feature, accessibility, and fast transfer capabilities. As a result, traditional banks have been compelled to adapt, creating their own digital platforms to stay relevant and accessible to their clients. However, this transition brings with it several risks, including greater exposure to cybersecurity vulnerabilities and declining standards of customer service.

Customer service remains the cornerstone of any successful business. As cliché as it may sound, clients are the main drivers of innovation, system upgrades, and even the decommissioning of outdated features. In the context of digital transformation, it is only natural for traditional businesses to establish a strong online presence to attract and retain customers. However, in the case of online banking, this digital presence has introduced new challenges—chief among them being the security vulnerabilities of these platforms. According to NICE Actimize (Haboken, 2023), there was a 92% increase in attempted global fraud in online banking in 2022, with the value of these attempts rising by 146% compared to 2021.

In the Philippines, the situation is even more concerning. TransUnion reports a 121% increase in fraudulent transactions and activities across financial institutions, fintech companies, and traditional banks. In 2024 alone, the suspected digital fraud rate in the Philippines reached 13.4% of all transactions—significantly higher than the global average of 5.3% (TransUnion, 2024).

Beyond fraudulent activities, another major issue in digital banking is the failure of fintech organizations to meet the core expectations of their industry. Digital banks are expected to deliver the same services as traditional banks but with greater convenience and speed—especially for payments, transfers, and financial products like loans. With that, the threat of fraud has created a heightened sense of alarm among users, making them more sensitive to any anomaly. One of the most common causes of user panic in digital banking is when money is deducted from their account but is not received by the intended recipient. This phenomenon is referred to as floating cash.

Floating cash refers to the situation where money is deducted from a user's account but not reflected in the recipient's account—often for hours or even days. There are several causes for this, including holdovers, which are delays in transaction processing due to factors such as weekend non-processing policies or institutional backlogs. Another type is transportation float, which can be caused by weather disruptions or air traffic delays (Segal, 2019). Regardless of the type, one thing remains constant: customers are dissatisfied whenever these delays occur, as they undermine confidence in the fintech application's basic functionality.

In a 2023 article, Jose Teodoro “TG” Limcaoco, President and CEO of BPI, referred to the bank's digital initiatives—such as its digital banking platform and Vybe, an e-wallet and rewards app—as the “best digital bank” and the “most reliable app” among competitors (Yu,

2023). However, by 2025, user sentiment appeared to shift. Complaints circulated on Reddit and other social media platforms, labeling Vybe as the “most unreliable application” for online banking (BPI Vybe - Most Unreliable Online Banking App, 2025).

Several users reported that transfers from Vybe to GCash or Maya were not completed, and funds appeared to vanish temporarily. One Reddit user in the r/DigitalBank subreddit stated, “My money's stuck in the transfer again, once 50k, another 30k. It takes almost a week before they return the money.” Such experiences were echoed by many others who reported persistent floating cash issues and difficulty transferring money using Vybe.

In light of these challenges, this project aims not only to resolve existing transactional issues but also to implement proactive measures to prevent them from occurring. By introducing real-time monitoring and immediate response mechanisms, the proposed solution seeks to improve operational reliability and enhance the user experience. Ultimately, a proactive and transparent system will help rebuild customer trust, reinforce BPI’s reputation, and solidify its position as a leader in digital banking. This approach will not only retain current Vybe users but also attract new ones who seek a trustworthy and efficient digital banking experience.

References:

BPI Vybe - Most unreliable online banking app. (2025). Reddit.com.

https://www.reddit.com/r/DigitalbanksPh/comments/1j0ai4g/bpi_vybe_most_unreliable_online_banking_app/?tl=en

Haboken, N. J. (2023). NICE Actimize 2023 Fraud Insights Report Reveals Attempted Fraud Transactions Have Increased By 92% Over Previous Year. NICE Systems.

<https://www.niceactimize.com/press-releases/nice-actimize-2023-fraud-insights-report-reveals-attempted-fraud-transactions-414/>

Segal, T. (2019). What Does Float Mean Financially? Investopedia. <https://www.investopedia.com/terms/f/float.asp> TransUnion Philippines. (2024, May 16).

TransUnion Study Finds Suspected Digital Fraud Rate in the Philippines Significantly Higher than Global Average. TransUnion Study Finds Suspected Digital Fraud Rate in the Philippines Significantly Higher than Global Average ; TransUnion Philippines. <https://newsroom.transunion.ph/transunion-study-finds-suspected-digital-fraud-rate-in-the-philippines-significantly-higher-than-global-average/>

Yu, L. (2023, August 2). BPI targets 50M customers in 3 to 5 years through digitalization. RAPPLER.

<https://www.rappler.com/business/bpi-targets-customers-through-digitalization/>

2. Solution Description

TRYBE (Trust Your Vybe)

"TRYBE" is an innovative, AI-powered digital banking solution designed to fundamentally address the persistent "floating cash" and unreliability issues plaguing the existing BPI Vybe app in the Philippines. Currently, users of banking apps, including BPI Vybe, often experience significant frustration when funds are debited but fail to reach their destination promptly, leading to what is commonly known as "floating cash." The crucial gap in existing banking applications, not just in the Philippines but globally, is the **absence of truly proactive, intelligent AI agents capable of autonomously detecting, predicting, and resolving**

such complex transactional anomalies in real-time. While some banks offer basic chatbots for FAQs or reactive customer support (like BPI's own BEA Chat, which primarily provides information and answers general inquiries), no major banking app yet features a sophisticated, interconnected system like TRYBE that intervenes *before* a customer is even aware of a potential issue, or resolves it within minutes.

At its core, "TRYBE" introduces a pioneering "AI Agent Architecture" – a suite of interconnected AI agents including the Discrepancy Detector (for identifying subtle transactional anomalies and stuck states), the Risk-Predictor (for forecasting potential delays or failures), the Auto-Reconciler (for initiating automated retries and rapid, 5-minute auto-reversals), the User Notifier (for delivering timely, empathetic, and culturally-aware updates), and the Ops Assistant (for intelligent human escalation). The entire system is orchestrated by the central Vybe Trust Manager, which collects insights from other agents, makes real-time flow decisions, and directly communicates with users via an integrated in-app chatbot. This project will also attempt to leverage simulated transaction environments and behavioral analytics (as a form of "KYC" without direct data access) to demonstrate its robust capabilities, further bolstered by the seamless and secure simulated integration of Passkeys for authentication.

What makes our solution unique and how it improves the Vybe app, promotes better customer experiences, and boosts the Vybe App's usage:

Our "TRYBE" solution stands out due to its proactive, intelligent, and user-centric approach to banking reliability – a paradigm shift from current industry standards. **Unlike existing banking applications that primarily rely on reactive customer service or manual intervention after a transaction has already failed, our AI Agent architecture continuously monitors and predicts potential issues.** It acts pre-emptively to prevent "floating cash" scenarios, ensuring that users in Makati and across the Philippines no longer face agonizingly long waits for resolution, which is a common source of negative sentiment on platforms like Reddit.

The TRYBE Manager is unique in its dual role as a sophisticated decision-maker and a direct, transparent communication channel. It fosters trust by providing immediate, clear, and culturally-sensitive updates to users *as issues are being resolved*, rather than forcing them to initiate contact. The commitment to **5-minute automated resolution** (via auto-retries and auto-reversals) for common floating fund issues is a game-changer, setting a new benchmark for digital banking efficiency and directly addressing the primary pain point of BPI Vybe users.

This **proof-of-concept AI customer service assistant, designed for autonomous issue resolution, represents a groundbreaking step for digital banking.** The modular and intelligent AI agent framework of TRYBE is **highly adaptable and can be readily applied to other BPI services.** Imagine extending this capability to:

- **Building on BEA Chat:** While BPI's current BEA Chat provides valuable information

and answers general inquiries, TRYBE's agents could seamlessly integrate with and significantly enhance its capabilities. Instead of just answering *about* problems, BEA could become an active participant in *resolving* them, leveraging the Discrepancy Detector and Auto-Reconciler to provide real-time status updates, initiate retries, or even process immediate reversals directly through the chat interface, transforming it from a reactive FAQ bot to a proactive problem-solver.

- **BPI Trade:** Proactively notifying users of potential order execution delays, automatically retrying failed trades within set parameters, or providing real-time, personalized alerts about unusual market movements affecting their portfolio. The Ops Assistant could flag complex trading discrepancies for immediate human review.
- **The Main BPI Mobile App:** Beyond Vybe, the same AI agents could monitor other transaction types like bill payments, loan disbursements, or scheduled transfers for anomalies, ensuring seamless operations across all banking functionalities. The User Notifier could provide proactive updates on branch appointment changes, document processing statuses, or even offer personalized financial advice based on detected spending patterns, truly transforming the customer experience across the entire BPI ecosystem.

Key Components and AI Agent Architecture

Our "TRYBE" will be powered by a suite of interconnected AI agents:

1. **"TRYBE Discrepancy Detector" (AI Agent Type: Discrepancy Detector)**
 - **Function:** Monitors *simulated* transaction ledgers and gateways for out-of-sync states. In a real-world scenario, this would involve listening to real-time API responses from BPI's internal systems and external partners (GCash, Maya, other banks).
 - **Prototype Implementation:**
 - **Simulated Transaction Log:** We will create a synthetic dataset mimicking transaction flows, including debits from a sender, credits to a recipient, and various "intermediate" states. This log will intentionally include scenarios where debits occur but corresponding credits are delayed or fail, simulating "floating cash."
 - **Logic-based "Gateway" Mimicry:** Simple scripts or functions will simulate the behavior of different bank APIs, introducing controlled delays or failures.
 - **Tooling/Stack:** Python (Pandas for data manipulation), SQL (for simulated ledger), Scikit-learn (for anomaly detection models).
2. **"TRYBE Risk-Predictor" (AI Agent Type: Risk-Predictor)**
 - **Function:** Forecasts which *simulated* transactions are likely to get stuck or delayed, focusing on the "floating cash" scenario. This allows for proactive intervention.
 - **Prototype Implementation:**
 - **Feature Engineering:** From the simulated transaction log, extract

features such as transaction amount, time of day, recipient type (e.g., "new recipient," "frequent recipient"), simulated network latency, historical success rates for similar transactions, etc.

- **Predictive ML Models:** Train classification models (e.g., Logistic Regression, Decision Trees, Random Forests, or even a simple Neural Network) on the simulated historical data to predict the likelihood of a transaction becoming "floating." The model will learn patterns associated with delays or failures.

- **Tooling/Stack:** Python (Scikit-learn, TensorFlow/Keras/PyTorch), Jupyter Notebooks for model training and evaluation.

3. "TRYBE Manager" (AI Agent Type: Manager and Flow Decider)

- **Function:** This central agent acts as the brain of the "TRYBE" system. It collects and analyzes the findings from the "TRYBE Discrepancy Detector" and the "TRYBE Risk-Predictor." Based on these insights, it makes real-time decisions on transaction flow (e.g., immediate release, temporary hold, initiate retry, trigger auto-reversal, or escalate for human review). It also serves as the primary direct communication channel with the user via an in-app chatbot.

- **Prototype Implementation (for students):**

- **Input Aggregation:** Simulate receiving data streams or outputs (e.g., "anomaly detected," "risk score X") from the Discrepancy Detector and Risk-Predictor.
- **Decision Logic Engine:** Implement a rule-based system or a state machine that processes these inputs to determine the appropriate action:
 1. **Normal Flow:** If no discrepancies and low risk, allow immediate processing.
 2. **Micro-Hold & Verification (5-minute target):** If moderate risk or minor discrepancy, initiate a short hold for automated checks or a simulated user confirmation (via Passkey).
 3. **Orchestrate Auto-Retry/Reversal:** Command the "TRYBE Auto-Reconciler" to execute these actions based on the severity and duration of the "floating" status.
 4. **Escalation:** If a transaction is highly complex, high-risk, or cannot be resolved automatically, it escalates the case to the "TRYBE Ops Assistant."
- **In-App Chatbot Interface:** Develop a simple chatbot within the simulated user interface. The Manager will generate and deliver proactive messages (e.g., "Your transfer is being reviewed for security," "Funds reversed due to delay") and respond to user queries regarding their transaction status, using templated responses or basic NLU/NLP (e.g., a small fine-tuned LLM or rule-based patterns for common questions). This handles the "in-app chatbot for problems" requirement.
- **Coordination:** Act as the central orchestrator, passing instructions to

the "TRYBE Auto-Reconciler" and "TRYBE User Notifier."

- **Tooling/Stack:** Python (for decision logic), LangChain (for agent orchestration and potentially basic NLU/LLM integration for the chatbot), Flask/Django (for the web UI where the chatbot resides), potentially a lightweight message queue (e.g., Redis Pub/Sub for inter-agent communication in prototype).

4. "TRYBE Auto-Reconciler" (AI Agent Type: Auto-Reconciler)

- **Function:** Triggers "retries" or "ledger updates" within the *simulated* environment when a "float" is detected by the Discrepancy Detector or predicted by the Risk-Predictor. In a real system, this would involve sending commands back to bank APIs.
- **Prototype Implementation:**
 - **Event-driven Workflows:** When a "float" is detected in the simulated log, a predefined workflow is triggered.
 - **Automated Retry Mechanisms:** This workflow will first attempt a *wsimulated auto-retry* of the transaction a limited number of times (e.g., 2-3 retries) with short, increasing delays (e.g., 10 seconds, 30 seconds, 1 minute). The system monitors for successful completion after each retry.
 - **Automated Reversal Logic:** If the transaction remains in "floating" status beyond a predefined threshold (e.g., 5 minutes for InstaPay-like transfers in the simulated environment) and the system determines it's unlikely to clear successfully after retries, it will **automatically initiate a full reversal of funds** back to the sender's account in the simulated ledger. This demonstrates the "5-minute resolution" goal effectively.
- **Tooling/Stack:** Python (simple scripting for workflows), potentially a lightweight workflow orchestration tool like Apache Airflow (or a simplified version if time permits), and LangChain (to orchestrate interactions between different AI "agents" in the prototype).

5. "TRYBE User Notifier" (AI Agent Type: User Notifier)

- **Function:** Informs the user via simulated chat/email if a "float" is happening or has been resolved, and provides in-app support.
 - Make sure it has a tone/culture-aware messages based on user demographics. Simple formula would be an apology, then explanation, and finally next steps.
- **Prototype Implementation:**
 - **Simulated User Interface:** A basic web interface or command-line interface that displays real-time transaction statuses.
 - **"GPT-based Assistant" Mimicry / In-app Chatbot:** Use simple Python strings or templates to generate user-friendly notifications (e.g., "Your transfer to [Recipient] is slightly delayed but we're working on it!", "Good news! Your transfer to [Recipient] has been completed."). This will include specific messages for **auto-reversals, clearly explaining why the funds were returned**. This can be expanded with a small

language model or even a simple rule-based chatbot for answering common FAQs about floating funds and transaction statuses directly within the UI.

- **Webhook Triggers (Simulated):** When a status changes in the simulated log (e.g., "floating," "retrying," "reversed," "completed"), trigger a "notification" to the user interface.
- **Post-Resolution Feedback Mechanism:** After a simulated transaction is successfully resolved (especially after it was in a "floating" state), the user interface will display a prompt or a quick survey asking for feedback on their experience with the resolution process. This demonstrates how the system would collect data for continuous improvement.

- **Tooling/Stack:** Python (Flask/Django for web UI, or just print statements for CLI), basic HTML/CSS/JS for simple UI.

6. "TRYBE Ops Assistant" (AI Agent Type: Ops Assistant)

- **Function:** Flags unresolved "float" cases to a *simulated* support or finance team.
 - Like a "Human Escalation Agent"
 1. For unresolved or high-impact issues, creates support ticket, schedules callback, pre-logs context.
- **Prototype Implementation:**
 - **Prioritized Task Queues:** A simple admin dashboard (web-based) that lists all "floating" transactions that the Auto-Reconciler couldn't resolve within its automated workflow (i.e., those that were reversed, or required manual intervention if the auto-reversal failed in a hypothetical scenario). These would be "escalated" cases in our simulated environment.
 - **"Copilot-like Agents" (Simplified):** For each unresolved case, the system could automatically generate a brief summary of the issue (e.g., "Transaction ID X to Y, Debited but no credit for 10 minutes, Risk-Predictor score: High, Auto-Reversal Initiated/Failed") to aid the simulated "Ops Team."
- **Tooling/Stack:** Python (Flask/Django for admin dashboard), basic HTML/CSS/JS.

Prototype Platform Summary (Mimicking BPI Vybe without Internal Access):

1. **A simulated transaction log:** As detailed under the Discrepancy Detector, this will be our synthetic data source for users, transfers, and delay/failure scenarios.
2. **A smart detection engine:** This encompasses the **Vybe Trust Discrepancy Detector** and **Vybe Trust Risk-Predictor**, leveraging ML and logic to flag risky or "floating" transactions.
3. **A managing agent: This is the Vybe Trust Manager.** It receives the outputs of the other agents and passes it on to the User Notifier and/or Admin Dashboard. It also

acts as a chatbot to the user.

4. **A user interface to track status, receive alerts, and simulate resolution:** This is where the **Vybe Trust User Notifier** shines, providing transparency to the user. Users can initiate simulated transfers and see their real-time (simulated) status updates. This also includes the **in-app chatbot** and **post-resolution feedback mechanism**.
5. **An admin dashboard for visualizing system-wide issues:** This is powered by the **Vybe Trust Ops Assistant**, providing an overview of all transactions and flagged "floating cash" incidents.

[OPTIONAL] Integrating KYC and Passkeys into the Prototype:

- **Behavioral KYC (via AI Agents):**
 - The **TRYBE Risk-Predictor** inherently incorporates "behavioral KYC." By analyzing simulated user patterns (e.g., usual amount, recipient, device fingerprint, time of day), it builds an implicit "user profile." Any deviation from this profile contributes to the transaction's risk score, acting as our "behavioral KYC" without needing direct access to actual BPI KYC data. This is how we "consider it but not entirely rely on it."
 - The prototype can include a simple module that, for each simulated user, tracks their "normal" behavior.
- **Passkeys (Simulated Implementation):**
 - While we can't integrate with actual hardware-backed passkeys, we can *simulate* the user experience.
 - **Login Simulation:** The prototype UI can present a "Login with Passkey" button. Upon clicking, it simulates successful biometric authentication, instantly logging the user in without a password. This demonstrates the seamless experience.
 - **Transaction Confirmation:** For "medium-risk" simulated transactions (as determined by the Risk-Predictor), the UI can pop up a "Confirm with Passkey" prompt. Clicking this button would simulate the user approving the transaction with their biometric, allowing the "Vybe Trust Auto-Reconciler" to proceed. This showcases the enhanced security and user control.

Implementation Plan

3. Target Audience

The primary users of the *TRYBE* are digital banking customers in the Philippines. The targeted users are young adults between the ages of 18 to 35, including university students, early-career professionals, gig economy workers, and small online business owners. Most of

them are urban-based, highly digital-literate, and frequently use mobile financial services such as GCash, Maya, and InstaPay for daily financial transactions. Given their digital fluency, these users expect seamless, reliable, and real-time financial experiences. They are also highly vocal online, particularly on platforms like Reddit and Twitter, where they share frustrations and app reviews. A key characteristic of this demographic is their low tolerance for delayed or failed transactions, especially when funds are deducted without confirmation of receipt, a scenario that directly leads to distrust in the app and public criticism. These users value fast resolution, clear communication, and trustworthiness from their banking platforms. Therefore, they are likely to respond positively to a solution like TRYBE, which addresses these concerns proactively through automation, transparency, and user-centric messaging.

4. Key Metrics for Success and Timeline

The development of the TRYBE prototype will be executed over a focused and time-bound schedule to balance technical depth with rapid iteration. Each milestone builds upon the previous to ensure a modular yet cohesive system by the end of the hackathon. The plan allows for early validation, iterative testing, and continuous feedback loops to de-risk integration and user-facing components. The optional final milestone offers a stretch opportunity for innovation in behavioral security and user control. Below is the proposed timeline and target duration for each phase of development:

Milestone 1: Synthetic Data Generation and Transaction Log Simulation

Timeline: Day 1 (4–6 hours)

The project begins with generating a high-fidelity synthetic transaction dataset and building a simulated transaction engine. This foundational work enables all downstream components to be tested under realistic, reproducible conditions. It includes anomaly injection such as floating cash scenarios and timestamp progression.

Success Metric: A dataset of at least 10,000 transactions, with a minimum of 500 floating cash scenarios, featuring diverse user behaviors and transaction types.

Milestone 2: Discrepancy Detector and Risk Predictor Development

Timeline: Day 1–Day 2 (8–10 hours)

Next, we develop and validate the core intelligence agents. The Discrepancy Detector identifies abnormal transaction flows, while the Risk Predictor forecasts which transactions are likely to result in user complaints due to stuck funds or anomalies.

Success Metric: At least 85% accuracy in detecting floating cash cases, and a Risk Predictor

AUC score of 0.8 or higher.

Milestone 3: Vybe Trust Manager and Auto-Reconciler Integration

Timeline: Day 2 (6 hours)

The system's orchestrator (the Vybe Trust Manager) is implemented to leverage insights from the agents and drive resolution actions. Paired with the Auto-Reconciler, this milestone ensures that stuck transactions can be automatically retried or reversed with minimal delay.

Success Metric: At least 70% of floating transactions resolved automatically within five simulated minutes, with <5% false positives on normal transactions.

Milestone 4: User-Facing Interface and Chatbot Functionality

Timeline: Day 2–Day 3 (6–8 hours)

The project then shifts toward user interaction. A notifier service and chatbot interface are added to inform and assist users with delayed transactions. These features are critical for simulating trust-building communication and minimizing support tickets.

Success Metric: 100% of flagged transactions notified in real time. At least 75% of chatbot inquiries resolved without escalation, with user satisfaction rated ≥ 4 out of 5.

Milestone 5: Ops Assistant and Admin Dashboard Deployment

Timeline: Day 3 (4 hours)

This milestone enables backend visibility and operational support. A dashboard will log unresolved or escalated cases with contextual summaries, enabling audit trails and mimicking what a real ops team would use.

Success Metric: Unresolved transactions must fall below 10% of the total volume. All escalated cases must be logged with transaction ID, risk score, failure cause, and resolution history.

Milestone 6 (Optional): Behavioral KYC and Simulated Passkey Integration

Timeline: Stretch Goal – Day 3 or as time allows (Optional, 4–6 hours)

This optional feature adds a security and personalization layer. It incorporates behavioral patterns into the Risk Predictor and includes UI support for simulated passkey approval (e.g., for high-value or risky transfers).

Success Metric: Behavioral patterns must inform the risk model, and simulated biometric

approvals must function in at least one key UI flow.

Final System Validation and Demo Preparation

Timeline: End of Day 3 (2–3 hours)

All components will be integrated into a complete, working prototype simulating a full transaction lifecycle. This includes initiation, anomaly detection, automated resolution, user notification, chatbot support, and operational logging.

Overall Success Indicator:

A seamless end-to-end simulation with all subsystems functioning in harmony. The workflow must operate reliably with realistic response times and demonstrate technical completeness and user-centered design.

Qualitative Success Indicators as an Effect of Those Above:

A. Financial Impact (Projected/Hypothetical): Reduction in Customer Support Costs:

- a. By automating resolutions and proactively communicating, we project a significant decrease in call center volume and associated operational costs for handling "floating cash" inquiries.
- b. Increased Transaction Volume: As trust and reliability improve, we anticipate users will conduct more transactions via BPI Vybe, leading to higher fee revenue and increased digital adoption.
- c. Customer Retention & Acquisition: A more reliable app directly translates to higher customer retention and a stronger appeal for new users, potentially reducing customer churn and acquisition costs.
- d. Reduced Fraud-Related Losses (Projected): While floating cash is not directly fraud, the enhanced vigilance of the "Vybe Trust Manager" and "Risk-Predictor" through behavioral KYC can proactively identify suspicious patterns, indirectly contributing to fraud prevention.

B. Reputation & Brand Image (Qualitative/Long-Term):

- a. Improved App Store Ratings: Higher ratings and more positive reviews on mobile app stores (Google Play, Apple App Store).
- b. Positive Social Media Sentiment: A shift from negative to positive mentions of "BPI Vybe" on social media platforms in the Philippines.

Technical Details

5. Technology Stack

Tentative list of the Technology Stack:

1. Presentation Layer

- a. Languages: TypeScript, Python
- b. Frontend: NextJS (Alternative: ReactJS)

2. Application Layer

- a. Backend: NextJS API, FastAPI

3. Agent Orchestration Layer

- a. Agent Frameworks: Google ADK (Alternative: LangChain/LangGraph)

4. Model Layer

- a. LLM Providers: Google Gemini (via GCP), OpenAI Models (via Azure), Hugging Face Transformers
- b. Traditional ML Frameworks: scikit-learn, XGBoost
- c. Deep Learning Frameworks: TensorFlow, PyTorch

5. Data Processing Layer:

- a. Data Manipulation: NumPy, pandas
- b. Visualization: Matplotlib

6. Storage Layer:

- a. Relational Database: PostgreSQL via Cloud SQL (Alternative: Firebase/Supabase/MongoDB)
- b. Vector Database: ChromaDB (if embeddings are needed)

7. Infrastructure and Deployment Layer

- a. Containerization: Docker
- b. Hosting: Google Cloud Run (Alternative: Vercel)
- c. Model Hosting: Vertex AI Studio

8. Version Control

- a. Source Control: Git and GitHub

9. Developer Tools and Collaboration

- a. IDEs and Notebooks: VS Code, Jupyter, Colab
- b. Planning: Miro
- c. Project Tracking: Notion or Jira
- d. Copilot: Claude Code
- e. Others: Google AI Studio, NotebookLM

6. Data Requirements

For the "TRYBE" prototype, our primary data requirement is a **comprehensive synthetic transaction dataset**. Since direct access to BPI's proprietary transaction logs and customer data is not feasible for a student project, we will generate this data to accurately mimic real-world digital banking scenarios, including the specific "floating cash" issues and other

anomalies. This synthetic dataset will be the foundation for training our AI agents, simulating the system's behavior, and demonstrating its capabilities.

Data We Need to Synthesize:

Our synthetic dataset will encompass the following key information for each simulated transaction:

1. Transaction Metadata:

- transaction_id: Unique identifier for each transaction.
- user_id: Simulated user initiating the transaction (allows for behavioral profiling).
- timestamp_initiated: Date and time when the user initiated the transfer.
- amount: Value of the transfer.
- transaction_type: E.g., "Bank to Bank (InstaPay)", "Bank to e-Wallet (GCash)", "Bank to e-Wallet (Maya)", "Internal BPI Transfer".
- recipient_type: E.g., "New Recipient", "Frequent Recipient".
- recipient_account_id: Simulated account number/wallet ID of the recipient.
- recipient_bank_name/e-wallet_name: The institution receiving the funds.
- device_id: Simulated device identifier of the user (for behavioral "KYC").
- location_coordinates: Simulated GPS coordinates of the transaction origin (for behavioral "KYC").
- simulated_network_latency: A random or patterned value representing network delay, which can contribute to "floating."

2. Transaction Status Progression (Simulated Journey):

- For each transaction, we need a sequence of timestamps and statuses to mimic its lifecycle. This is crucial for detecting "floating" states.
- status_timestamp_1: Timestamp for "Initiated"
- status_1: "Initiated"
- status_timestamp_2: Timestamp for "Debit Confirmed (BPI)"
- status_2: "Debit Confirmed (BPI)"
- status_timestamp_3: Timestamp for "Processing (Recipient Bank/e-Wallet)" (optional intermediate state)
- status_3: "Processing (Recipient Bank/e-Wallet)"
- status_timestamp_4: Timestamp for "Credit Confirmed (Recipient)" or "Failed/Reversed"
- status_4: "Credit Confirmed (Recipient)", "Failed (Reason)", or "Reversed (Reason)"
- expected_completion_time: A calculated ideal time for the transaction to complete based on type/recipient.

3. Anomaly/Failure Indicators (for ML training and simulation):

- is_floating_cash: Boolean (True/False) indicating if this transaction *should* enter a floating state in the simulation.
- floating_duration_minutes: If is_floating_cash is True, how long does it remain

floating?

- is_fraudulent_attempt: Boolean (True/False) for simulated fraud attempts (e.g., unusual patterns detected by Risk-Predictor).
- is_cancellation: Boolean (True/False) indicating if the user cancelled the transaction.
- is_retry_successful: Boolean (True/False) indicating if an auto-retry would resolve it.
- manual_escalation_needed: Boolean (True/False) indicating if it's an unresolvable case.

How We Plan to Create the Synthetic Dataset:

We will programmatically generate this dataset using Python, leveraging libraries like pandas for data structuring and numpy for random number generation and statistical distributions.

The process will involve several steps:

1. Define User Profiles (Simulated Behavioral KYC):

- Create a set of N simulated user_ids.
- For each user, define a "normal" behavioral profile:
 - typical_transaction_amount_range: (min, max)
 - typical_transaction_frequency: (e.g., transactions per day/week)
 - most_frequent_recipient_types: (e.g., 70% GCash, 20% other banks, 10% Maya)
 - primary_device_ids: A list of device IDs usually associated with the user.
 - usual_transaction_hour_ranges: (e.g., 9 AM - 5 PM)
 - Introduce a small percentage of "anomalous" users who occasionally perform unusual transactions to provide data for anomaly detection.

2. Generate Transaction Streams:

- Iterate through a simulated timeline (e.g., over several months or a year).
- For each time step, randomly select a user_id based on their activity frequency.
- Generate amount, transaction_type, recipient_type, recipient_account_id (creating new ones if "new recipient"), device_id, and location_coordinates based on the selected user's "normal" profile, with controlled deviations to introduce anomalies.

3. Simulate Transaction Lifecycle and Inject Anomalies:

- For each generated transaction, simulate its progression through various statuses (initiated, debit confirmed, processing, credit confirmed).
- **Inject "Floating Cash" Scenarios:** For a predefined percentage of transactions (e.g., 5-10% to reflect the problem statement), intentionally introduce delays between debit confirmed and credit confirmed (or make credit confirmed never appear), creating the "floating" state. These delays will be varied to simulate different severity levels.

- **Simulate Other Anomalies:**
 - **Failed Transactions:** Randomly mark some transactions as "failed" at various stages, with different "failure reasons" (e.g., "network error," "recipient invalid," "system timeout").
 - **Cancellations:** Simulate user cancellations at different points in the transaction lifecycle.
 - **Unusual Patterns:** Introduce sequences of multiple failed attempts, or high-value transfers to new recipients from unusual devices/locations to train the Risk-Predictor.
- **Simulate Resolution:** For floating transactions, some will resolve naturally after a delay, others will be marked for "auto-retry," and a subset will be flagged for "auto-reversal" (as dictated by our Better Vybe Auto-Reconciler logic).

4. Data Storage:

- The generated dataset will be stored in a tabular format (e.g., CSV files or a lightweight SQLite database) to serve as the "Simulated Transaction Log" for the prototype. This will allow the Discrepancy Detector and Risk-Predictor to query and analyze the data as if it were coming from a live system.

Additional Information

7. Challenges and Risks

- **Technical Challenges**

- Synthetic Data Generation:
 - Generating truly realistic synthetic data that accurately mirrors the complexity, volume, and nuances of real-world banking transactions (including the subtle patterns leading to "floating cash") is difficult.
- AI Model Training and Performance:
 - Achieving high accuracy for the Discrepancy Detector and Risk-Predictor on limited, synthetic data might not translate well to real-world scenarios. Overfitting to the synthetic data or failing to generalize to unforeseen patterns are significant risks.
- Single Point of Failure (TRYBE Manager):
 - The TRYBE Manager acts as the central brain. If it fails or has logical errors, the entire system's ability to detect, predict, and resolve "floating cash" issues could be negatively affected.
- Hallucination
 - All sub-agents with an LLM brain are prone to hallucination. Namely:
 - Notifier (Chatbot that may give incorrect chats)
 - Manager (May decide and pass information incorrectly)

- Ops (May give incorrect messages to the admin)
- **Data Ethics**
 - Algorithmic Bias:
 - If the synthetic data (or real data in a future deployment) contains inherent biases (e.g., certain transaction types or user demographics are disproportionately associated with "floating cash" or perceived higher risk), the Risk-Predictor could learn and perpetuate these biases. This could lead to certain user groups experiencing more delays, more frequent reversals, or more stringent "Passkey" prompts, leading to unfair treatment or discrimination. For example, new users or users transacting with less common recipients might be unfairly flagged.
- **Regulatory Compliance (for Future Deployment)**
 - Digital banking solutions in the Philippines are subject to regulations from the Bangko Sentral ng Pilipinas (BSP) concerning data privacy, consumer protection, and operational resilience (Bangko Sentral ng Pilipinas, 2020). While a prototype isn't bound by this, acknowledging these future considerations is important.

References

Bangko Sentral ng Pilipinas. (2020). Guidelines on the Establishment of Digital Banks.
<https://www.bsp.gov.ph/Regulations/Issuances/2020/c1105.pdf>
 Bangko Sentral ng Pilipinas. (n.d.). WHAT IS a PAYMENT SYSTEM?
https://www.bsp.gov.ph/PaymentAndSettlement/FAQ_PhilPaSS.pdf

Reminders

July 15, 2025 at 11:59 PM - Submission of initial proposal (PDF format)

Kindly fill in this template, download as PDF and submit it on or before the deadline. The deadline for the Initial Proposal in PDF format is July 15, 2025 11:59 PM. We will provide an Initial Submission Google Form so you can submit this document in PDF format there.

August 25, 2025 at 11:59 PM - Submission of final proposal (Video presentation of project, slide deck used or demo links used in video presentation, and project paper containing documentation)

More info on the video duration and rubrics will be shared after July 15, 2025.